5.5 sync.WaitGroup в GoLang

Ключевые моменты

`sync.WaitGroup` позволяет главной горутине (`main`) или другой горутине дождаться завершения группы горутин.

Решает проблему синхронизации, обеспечивая надежное ожидание без лишних накладных расходов.

До `WaitGroup` использовались каналы, мьютексы или `time.Sleep` , что приводило к сложному коду или ненадежным результатам.

Примеры кода демонстрируют использование `WaitGroup` и альтернативные подходы, подчеркивая его преимущества.

Почему появился `sync.WaitGroup`?

В Go горутины — легковесные потоки, которые позволяют выполнять задачи параллельно. Однако главная горутина (`main`) завершается, не дожидаясь завершения других горутин, что может привести к потере результатов или преждевременному завершению программы. Например, если горутина выполняет длительную задачу, `main` может выйти раньше, чем задача завершится.

`sync.WaitGroup` был введен в стандартной библиотеке Go (`sync`) для предоставления простого и надежного способа координации горутин. Он позволяет указать, сколько горутин нужно дождаться, и блокировать выполнение до их завершения. Это упростило написание конкурентных программ по сравнению с более сложными или ненадежными методами.

Цель

`WaitGroup` появился, чтобы:

- Обеспечить декларативный способ ожидания завершения горутин.
- Устранить необходимость в сложной синхронизации через каналы или мьютексы. Избежать ненадежных решений, таких как `time.Sleep`.
- Сделать код чище и менее подверженным ошибкам.

Какую проблему решает `sync.WaitGroup` ?

Проблема

В конкурентных программах нужно дождаться, пока все горутины завершат свою работу, прежде чем продолжить выполнение (например, вывести результаты или завершить программу). Без механизма синхронизации главная горутина может завершиться раньше, что приводит к:

- Потере результатов, так как горутины не успевают завершить работу.
  Непредсказуемому поведению программы.
- Сложности в отладке, так как программа завершается преждевременно.

Пример проблемы

Программа, где `main` завершается раньше горутин:

```
package main

 import (
  "fmt"
  "time"
)

func worker(id int) {
fmt.Printf("Работник %d начал\n", id)
   time.Sleep(time.Second) // Имитация работы
   fmt.Printf("Работник %d завершил\n", id)
}

func main() {
   go worker(1) go worker(2)
   fmt.Println("main завершился")
}
```

Вывод

```
main завершился
```

Проблема: Горутины не успевают вывести сообщения, так как `main` завершается сразу. Запуск может иногда показывать частичный вывод, но результат непредсказуем.

Решение с `WaitGroup`

`sync.WaitGroup` решает эту проблему, позволяя `main` дождаться завершения всех горутин:

```
package main

import (
 "fmt"
 "sync"
 "time"
)

func worker(id int, wg *sync.WaitGroup) {
   defer wg.Done() // Уменьшает счетчик при завершении
   fmt.Printf("Работник %d начал\n", id) time.Sleep(time.Second)
   fmt.Printf("Работник %d завершил\n", id)
}

func main() {
   var wg sync.WaitGroup
   wg.Add(2) // Устанавливаем счетчик на 2 горутины

   go worker(1, &wg) go worker(2, &wg)

   wg.Wait() // Блокирует, пока счетчик не станет 0
   fmt.Println("main завершился")
}
```

Вывод

```
Работник 1 начал
Работник 2 начал
Работник 1 завершил
Работник 2 завершил main завершился
```

Объяснение

- `wg.Add(2)` устанавливает счетчик горутин.
- `wg.Done()` уменьшает счетчик, когда горутина завершает работу.
- `wg.Wait()` блокирует `main` , пока счетчик не станет 0.

Как решали проблему до появления `WaitGroup`?

До введения `sync.WaitGroup` в Go (или в других языках без аналогичного механизма) разработчики использовали альтернативные подходы, которые были менее удобными и более подверженными ошибкам. Основные методы включали:

1. **Каналы для сигнализации**:

   Горутины отправляли сигнал о завершении через канал.
   Главная горутина считывала сигналы, чтобы определить, когда все горутины завершились.

2. **Счетчики с мьютексами**:

   Использовалась общая переменная-счетчик, защищенная мьютексом, для отслеживания завершенных горутин.

   Главная горутина проверяла счетчик, чтобы дождаться завершения.

3. **time.Sleep**:

   Добавлялась задержка в `main`, чтобы дать горутинам время завершиться.

   Ненадежный метод, так как время выполнения горутин неизвестно.

Пример 1: Использование каналов

Каналы использовались для отправки сигналов о завершении:

```
package main

import (
  "fmt"
  "time"
)

func worker(id int, done chan bool) {
  fmt.Printf("Работник %d начал\n", id)
  time.Sleep(time.Second)
  fmt.Printf("Работник %d завершил\n", id)
  done <- true // Сигнал о завершении
}

func main() {
   done := make(chan bool, 2) // Буферизованный канал
   go worker(1, done)
   go worker(2, done)

   // Ожидаем завершения обеих горутин
   <-done
   <-done
   fmt.Println("main завершился")
}
```

Вывод

```
Работник 1 начал

Работник 2 начал

Работник 1 завершил
Работник 2 завершил main завершился
```

Недостатки:

- Требуется вручную считывать сигналы для каждой горутины.
- Для большого числа горутин код становится громоздким.
- Необходимо заранее знать количество горутин или использовать дополнительные структуры.

Пример 2: Счетчик с мьютексом

Счетчик защищался мьютексом для отслеживания завершенных горутин:

```
package main

import (
  "fmt"
  "sync"
  "time"
)

func worker(id int, counter *int, mu *sync.Mutex, done chan bool) {
   fmt.Printf("Работник %d начал\n", id)
   time.Sleep(time.Second) fmt.Printf("Работник %d завершил\n", id)
   mu.Lock()
   *counter--
   if *counter == 0 {
   done <- true }
   mu.Unlock()
}

func main() {
   var mu sync.Mutex counter := 2
   done := make(chan bool)
   go worker(1, &counter, &mu, done)
   go worker(2, &counter, &mu, done)

   <-done // Ожидаем сигнала
   fmt.Println("main завершился")
}
```

Вывод

```
Работник 1 начал

Работник 2 начал

Работник 1 завершил
Работник 2 завершил

main завершился
```

Недостатки

- Код сложнее из-за мьютекса и канала.
- Риск гонки данных без правильной синхронизации.
- Требует дополнительной логики для проверки счетчика.

Пример 3: Использование `time.Sleep`

Ненадежный метод с задержкой:

```
package main

 import (
  "fmt"
  "time"
)

func worker(id int) {
  fmt.Printf("Работник %d начал\n", id)
  time.Sleep(time.Second)
  fmt.Printf("Работник %d завершил\n", id)
}

func main() {
   go worker(1) go worker(2)
   time.Sleep(2 * time.Second) // Произвольная задержка
   fmt.Println("main завершился")
}
```

Вывод (зависит от времени)

```
Работник 1 начал
Работник 2 начал
Работник 1 завершил
Работник 2 завершил
main завершился
```

Недостатки:

- Задержка (`time.Sleep`) может быть недостаточной, если горутины работают дольше.
- Избыточная задержка замедляет программу.
- Ненадежно для реальных приложений.

Преимущества `sync.WaitGroup`

`WaitGroup` решает проблемы предыдущих методов:

- **Простота**: Нет необходимости вручную управлять каналами или мьютексами.
- **Гибкость**: Поддерживает динамическое добавление горутин через `Add`.
- **Надежность**: Гарантирует ожидание всех горутин без предположений о времени выполнения.
- **Эффективность**: Минимальные накладные расходы по сравнению с каналами или мьютексами.

Пример продвинутого использования

Динамическое добавление горутин:

```
package main

import (
 "fmt"
 "sync"
 "time"
)

func worker(id int, wg *sync.WaitGroup) { defer
   wg.Done()
   fmt.Printf("Работник %d начал\n", id)
   time.Sleep(time.Second) fmt.Printf("Работник %d
   завершил\n", id)
}

func main() {
   var wg sync.WaitGroup
   workers := 5

   for i := 1; i <= workers; i++ { wg.Add(1)
      go worker(i, &wg)
   }

   wg.Wait()
   fmt.Println("Все работники завершили работу")
}
```

Вывод

```
Работник 1 начал
Работник 2 начал
Работник 3 начал
Работник 4 начал
Работник 5 начал
Работник 1 завершил
Работник 2 завершил
Работник 3 завершил
Работник 4 завершил
Работник 5 завершил

Все работники завершили работу
```

Объяснение: `wg.Add(1)` вызывается для каждой горутины перед ее запуском, а `wg.Wait()`блокирует до завершения всех.

Типичные ошибки при использовании `WaitGroup`

1. **Вызов** `Done` **без** `Add`

```
package main

import(
  "fmt"
  "sync"
)

func main() {
   var wg sync.WaitGroup go func() {
      fmt.Println("Работа")
      wg.Done() // Паника: отрицательный счетчик
   }()
   wg.Wait()
}
```

Решение: Всегда вызывать `wg.Add(1)` перед `wg.Done()`.

2. **Вызов `Add` после `Wait`**

```
package main

import (
  "fmt"
  "sync"
  "time"
)

func main() {
   var wg sync.WaitGroup go func() {
      time.Sleep(time.Second)
      wg.Add(1) // Вызывается после Wait
      fmt.Println("Работа")
      wg.Done()
   }()
   wg.Wait() // Зависает, так как Add вызывается поздно
}
```

Решение: Вызывать `Add` до `Wait` или в начале программы.

- **Внутренняя реализация**: `WaitGroup` использует счетчик и семафор для отслеживания горутин. `Add` увеличивает счетчик, `Done` уменьшает, а `Wait` блокирует, пока счетчик не станет 0.
- **Потокобезопасность**: `WaitGroup` безопасен для конкурентного использования, но требует правильного порядка вызовов (`Add` → `Done`).
- **Ограничения**: Не поддерживает повторное использование после `Wait` без явного сброса счетчика через `Add`.
- **Альтернативы**: Каналы или контекст (`context`) могут использоваться для более сложной синхронизации.

Ключевые источники

- Go by Example: WaitGroup
- Golangbot: WaitGroup
- Go Documentation: sync.WaitGroup
- Go 101: Concurrency Synchronization